

BAB 5 State, Props dan Hooks

5.1 Tujuan Pembelajaran

1. Memahami konsep state dan fungsinya dalam pengelolaan data dinamis pada aplikasi React Native.
2. Memahami konsep props sebagai mekanisme pengiriman data antar komponen.
3. Menjelaskan peran Hooks, khususnya useState dan useEffect, dalam pengembangan komponen berbasis fungsi.
4. Menerapkan state, props, dan hooks dalam studi kasus aplikasi React Native sederhana.
5. Mengembangkan komponen yang lebih interaktif dan responsif terhadap input pengguna.

5.2 Dasar Teori

5.2.1 State

State merupakan mekanisme untuk menyimpan dan mengelola data yang bersifat dinamis di dalam sebuah komponen. Data yang disimpan sebagai state dapat berubah akibat interaksi pengguna atau proses tertentu, dan setiap perubahan state akan secara otomatis memperbarui tampilan (*re-render*).

Pada React Native modern, state dikelola menggunakan *Hook* **useState**. Dalam mengimplementasi useState, ada dua tipe yang bisa digunakan, pertama ialah dalam bentuk Destructuring dan Non Destructuring.

A. Penggunaan Destructuring:

Format ini menggunakan **Array Destructuring** dari JavaScript ES6. Dengan bentuk struktur *useState()* seperti `[nilai_state, function_state]`.

Contoh:

```
const [count, setCount] = useState(0);
const [name, setName] = useState("");
const [isActive, setIsActive] = useState(false);
const [listBook, setListBook] = useState([]);
```

Dalam penggunaan useState pattern yang dipakai ialah:

- State variable: *namaVariable*
- Setter function: *setNamaVariable* (dengan prefix "set")

```
const [count, setCount] = useState(0)
```

Annotations in the diagram:

- `count`: State variable (untuk baca nilai)
- `setCount`: Setter function (untuk update)
- `useState`: Hook function
- `0`: Initial value

Gambar 5.1 Pengidentifikasian useState pada React

B. Penggunaan Non Destructuring:

Untuk menggunakan Non Destructuring, dapat dilakukan dengan cara membuat sebuah variable yang mengembalikan nilai dalam bentuk array dan menginisialisasinya setiap elemen dengan index.

```
const countState = useState(0)
const count = countState[0]
const setCount = countState[1]
```

Dari code diatas menyatakan bahwa:

- variable countState memiliki nilai hook ialah 0 dalam bentuk array
- variable count digunakan untuk mengambil nilai index ke-0 dari countState
- variabel setCount akan mengambil nilai index ke-1 dari countState

C. Perbandingan antara Destructuring dengan Non Destructuring

Dari penjelasan sebelumnya dapat disimpulkan bahwa penggunaan *useState Destructuring* lebih baik digunakan sebagai media pengolahan data, kenapa lebih baik berikut table ringkasan dari kedua format *useState*:

Non Destructuring	Destructuring
3 baris per state	1 baris per state
Kurang readable	Mudah dibaca
Tidak idiomatis (tidak mengikuti best practice React)	Mengikuti best practice React, singkat dan jelas
Sulit maintain untuk banyak state	Lebih maintainable

Table 5.1 Perbedaan antara Destructuring dan Non-Destructuring

Oleh karenanya selalu gunakan destructuring untuk *useState()* dan hooks lainnya!

Contoh penggunaan:

```
1 import React, { useState } from 'react'
2 import { View, Text, Button } from 'react-native'
3 import { SafeAreaView } from 'react-native-safe-area-context'
4
5 export default function Latihan5() {
6   const [count, setCount] = useState(0)
7   return (
8     <SafeAreaView style={{ flex:1 }}>
9       <Text>Count: { count}</Text>
10      <View style={{ margin: 10, flexDirection:'row', alignItems:'center', justifyContent:'space-between' }}>
11        <Button title="Increment (+)" onPress={() => setCount(count + 1)} />
12        <Button title="Decrement (-)" onPress={() => setCount(count - 1)} />
13      </View>
14    </SafeAreaView>
15  )
16 }
```

Penjelasan:

1. Import Statements

```
import React, { useState } from 'react'
```

Mengimport React dan hook *useState* untuk state management

2. State Management dengan useState

- **useState(0)**: Menginisialisasi state dengan nilai awal 0
- **count**: Variable yang menyimpan nilai state saat ini
- **setCount**: Function untuk mengupdate nilai count
- Setiap kali *setCount* dipanggil, komponen akan re-render dengan nilai baru

3. Increment Button

- Ketika ditekan, memanggil `setCount(count + 1)`
- Menambah nilai count sebesar 1

4. Decrement Button

- Ketika ditekan, memanggil `setCount(count - 1)`
- Mengurangi nilai count sebesar 1

5.2.2 Props

Props merupakan mekanisme untuk mengirim data dari komponen induk (*parent*) ke komponen anak (*child*). Data yang dikirim melalui props bersifat satu arah (*one-way data flow*) dan tidak dapat diubah oleh komponen penerima.

Agar penggunaan props lebih mudah dipahami, berikut format umum implementasi props dalam React Native.

Komponen Parent:

```
const Latihan5 = () => {
  const book = {
    title: "Mobile Programming with React Native",
    author: "Septian Cahyadi",
    year: 2005,
  };

  return (
    <View>
      <Text>This is a Parent Component</Text>
      <SelectedBook book={book} />
    </View>
  )
}
```

Komponen Child:

```
const SelectedBook = ({ book }) => {
  return (
    <View>
      <Text>Title: {book.title}</Text>
      <Text>Author: {book.author}</Text>
      <Text>Year: {book.year}</Text>
    </View>
  )
}
```

A. Function Declaration dan Data Object

- Nama RFC *Latihan5* menunjukkan ini adalah komponen induk (parent)
- Dan komponen ini tidak menerima props (parameter kosong)
- Membuat object literal bernama book
 - Berisi 3 properti:
 - title: String - judul buku
 - author: String - nama penulis
 - year: Number - tahun terbit
- Object tersebut akan di-pass sebagai props ke child component

B. Menghubungkan dengan child component

- Nama RFC *SelectedBook* menunjukkan ini adalah komponen anak (child)
- Data komponen ini menerima props bernama 'book'
- Data yang diterima props ialah bertipe object

C. Cara kerja Props

1. Parent mengirimkan data:

```
<SelectedBook book={book} />
```

↑ ↑
prop name; prop value

2. Child menerima data:

```
const SelectedBook = ({ book }) => {
```

↑
destructuring props

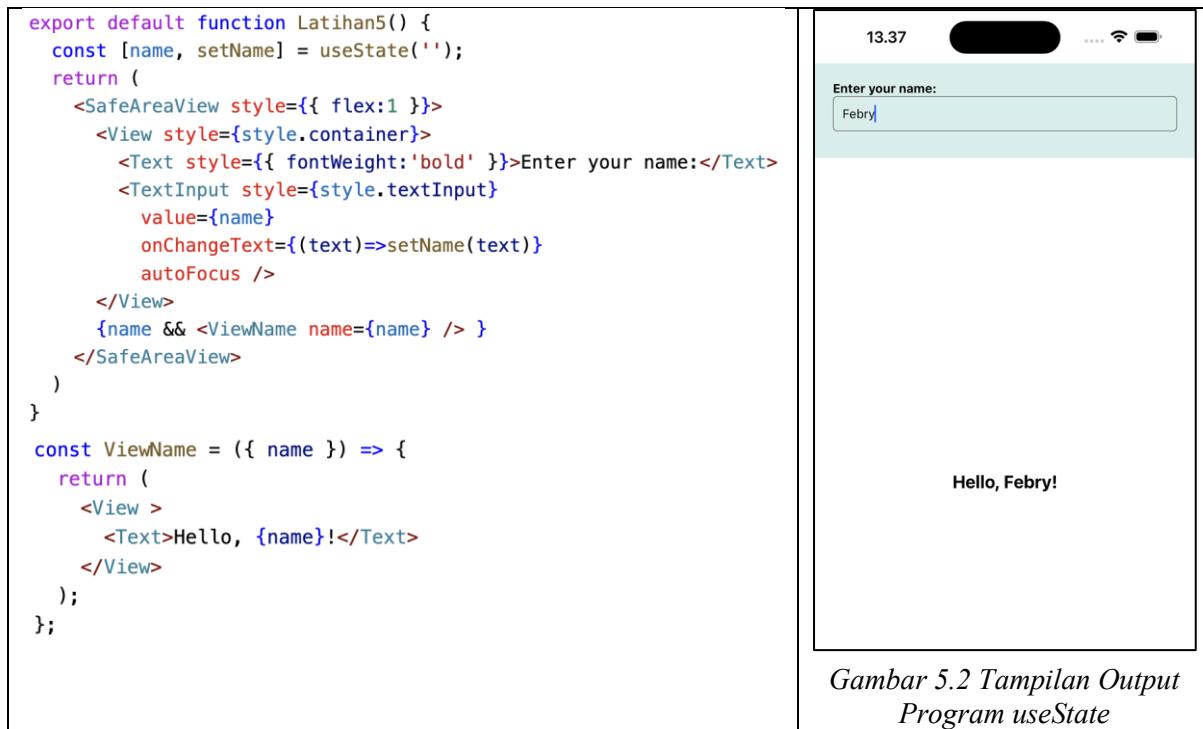
3. Child menggunakan data:

```
<Text>Title: {book.title}</Text>
```

↑
akses properti dari props

Berikut adalah contoh penggunaan kombinasi antara state dan props:

Code	Output
------	--------



Penjelasan:

- Deklarasi state management untuk mengahandle inputan dalam bentuk string yang bernilai kosong:

```
const [name, setName] = useState("");
```

- Controlled component pattern

```

<TextInput style={style.textInput}
  value={name}
  onChangeText={(text)=>setName(text)}
  autoFocus />

```

Cara kerja:

```

User ketik "F"
  ↓
onChangeText triggered
  ↓
setName("F") dipanggil
  ↓
State name berubah: "" → "F"
  ↓
Component re-render
  ↓
TextInput value={name} → value="F"
  ↓
UI update: TextInput menampilkan "F"

```

- Conditional Rendering

```
{name && <ViewName name={name} /> }
```

↑ ↑

prop value

Ini adalah *Short-Circuit Evaluation* (Logical AND), mengecek kondisi HANYA JIKA state name memiliki nilai atau tidak kosong maka akan menampilkan RFC ViewName dengan mengirim properti bernama name

5.2.3 Hooks

Hooks merupakan fitur pada React yang memungkinkan penggunaan *state*, *lifecycle*, dan optimasi performa di dalam komponen berbasis fungsi (*Functional Component*). Dengan *Hooks*, pengembang tidak lagi bergantung pada *Class Component*, sehingga kode menjadi lebih ringkas, mudah dibaca, dan mudah dirawat.

Hooks bekerja berdasarkan prinsip:

- Hanya dapat digunakan di komponen fungsi (RFC)
- Dipanggil di level teratas komponen (bukan di dalam kondisi atau loop)
- Mengandalkan dependency array untuk mengontrol kapan sebuah proses dijalankan
- Tidak dipanggil didalam *Selections*, *Repetitions*, ataupun fungsi biasa
- Urutan pemanggilan *Hooks* harus konsisten

Hooks yang umum digunakan pada React Native:

- *useState*: Mengelola state
- *useEffect*: Menangani efek samping (lifecycle)
- *useMemo*: Optimasi performa dalam bentuk memoizing
- *useContext*: Berbagi data global (lanjutan)
- *useRef*: Penyimpanan nilai yang tidak perlu dirender

A. Hook *useEffect*

Jika diibaratkan, React adalah sebuah panggung sandiwara, *useEffect* adalah kru di balik layar yang bertugas membereskan panggung setelah aktor selesai beraksi atau menyiapkan properti sebelum adegan dimulai.

Secara teknis, *useEffect* adalah *Hook* dalam React yang memungkinkan kita melakukan *side effects* (efek samping) di dalam function component.

Dalam pemrograman, side effect adalah segala sesuatu yang "keluar" dari alur murni render UI. Contohnya:

- Mengambil data dari API (*Fetching data*).
- Mengubah DOM secara manual (misalnya mengganti judul dokumen).
- Mengatur timer seperti *setTimeout* atau *setInterval*.
- Berlangganan (Subscription) ke sumber data eksternal.

Struktur dasar *useEffect* terdiri dari dua bagian utama: fungsi efek dan array ketergantungan (*dependency array*).

```
useEffect(() => {
  // 1. Kode efek di sini (Logika yang ingin dijalankan)

  return () => {
    // 2. Cleanup function (Opsional: dijalankan sebelum komponen di-unmount)
  };
}, [dependency]); // 3. Dependency Array (Menentukan kapan efek berjalan)
```

Cara `useEffect` berjalan sangat bergantung pada apa yang kamu masukkan ke dalam array ketergantungan di bagian akhir:

Dependency Array	Kapan Efek Berjalan?
Tanpa Array <code>useEffect(() => {...})</code>	Berjalan setiap kali komponen melakukan render (sangat jarang digunakan karena boros performa).
Array Kosong <code>useEffect(() => {...}, [])</code>	Hanya berjalan satu kali setelah render pertama (mirip <code>componentDidMount</code>). Cocok untuk ambil data API.
Ada Isinya <code>useEffect(() => {...}, [prop, state])</code>	Berjalan saat pertama kali render DAN setiap kali nilai <code>prop</code> atau <code>state</code> tersebut berubah.

Tabel 5.2 Penggunaan `useEffect` pada React

Kegunaan Utama:

- Sinkronisasi dengan Data Luar: Menghubungkan aplikasi kamu dengan API atau database.
- Manajemen Event Listener: Menambahkan event seperti scroll atau resize pada jendela browser.
- Pembersihan (Cleanup): Menghapus timer atau membatalkan langganan agar tidak terjadi kebocoran memori (memory leak).

B. Hook `useMemo`

Jika `useEffect` adalah kru di balik layar, maka `useMemo` adalah asisten pribadi yang memiliki ingatan sangat tajam.

Dalam React, `useMemo` adalah *Hook* yang digunakan untuk memoarisasi (menyimpan hasil perhitungan) agar tidak perlu dihitung ulang setiap kali komponen melakukan render, kecuali jika ada data pendukung yang berubah.

Secara default, React akan menjalankan ulang semua logika di dalam fungsi komponen setiap kali terjadi *re-render* (misalnya saat ada *state* atau *props* yang berubah). Jika memiliki perhitungan yang sangat berat (seperti memproses ribuan data atau algoritma matematika yang rumit), menjalankannya di setiap render akan membuat aplikasi terasa lambat atau "lag". `useMemo` hadir untuk mencegah hal tersebut.

```
const hasilMemo = useMemo(() => {
  // 1. Logika perhitungan yang berat
  return hitungSesuatuYangMahal(a, b);
}, [a, b]); // 2. Dependency Array
```

- Fungsi *hasilMemo*: Fungsi yang mengembalikan nilai yang ingin disimpan.
- *Dependency Array*: Daftar variabel yang dipantau. Jika a atau b berubah, React akan menghitung ulang nilainya. Jika tidak, React akan langsung mengambil hasil simpanan sebelumnya.

Kapan Sebaiknya TIDAK menggunakan *useMemo*:

- Jangan gunakan *useMemo* untuk setiap variabel kecil
- *Overhead*: *useMemo* sendiri butuh memori untuk menyimpan nilai. Jika perhitungannya sangat ringan (seperti $1 + 1$), penggunaan *useMemo* justru malah membebani performa.
- Keterbacaan: Kode jadi lebih kompleks. Gunakan hanya jika benar-benar merasakan adanya penurunan performa atau jika nilai tersebut digunakan sebagai dependensi di Hook lain.

C. Hook *useContext*

Jika *useEffect* adalah kru panggung dan *useMemo* adalah ingatan asisten, maka *useContext* adalah sistem pengeras suara (*broadcast*) di seluruh gedung pertunjukan.

Dalam React, data biasanya dikirim dari atas ke bawah (*parent to child*) melalui props. Namun, jika komponennya sangat dalam, kita akan mengalami masalah yang disebut *Prop Drilling*, yaitu melewati data melalui komponen tengah yang sebenarnya tidak butuh data tersebut. *useContext* hadir untuk memintas jalur itu.

useContext memungkinkan komponen untuk "subscribe" ke sebuah data global tanpa harus menerima data tersebut lewat props.

Cara penggunaan:

1. Membuat context (*createContext*)

```
const TemaContext = createContext('light');
```

Biasanya dibuat di file terpisah atau di bagian atas komponen induk.

2. Menyediakan context (*Provider*)

Bungkus komponen yang membutuhkan data dengan *.Provider*. Semua komponen di dalamnya (dan anak-anaknya) sekarang punya akses ke nilai tersebut.

```
<TemaContext.Provider value="dark">
  <ChildComponent />
</TemaContext.Provider>
```

3. Mengonsumsi context (*useContext*)

```
const tema = useContext(TemaContext);
```

Di dalam komponen anak, panggil Hook ini untuk mengambil nilainya.

Kapan Harus Menggunakan *useContext*:

Gunakan *useContext* untuk data yang bersifat global atau digunakan oleh banyak komponen di tingkat yang berbeda:

- Tema: Mode gelap/terang.

- Autentikasi: Data pengguna yang sedang login (username, token).
- Bahasa: Lokalisasi (Inggris, Indonesia, dll).
- Keranjang Belanja: Data produk yang dipilih di aplikasi e-commerce.

D. Hook `useRef`

Jika `useContext` adalah pengeras suara, maka `useRef` adalah sebuah kotak penyimpanan pribadi yang tidak terlihat oleh penonton.

Secara sederhana, `useRef` adalah *Hook* yang mengembalikan sebuah objek dengan satu properti saja: `{ current: ... }`. Keunikan utamanya adalah: mengubah nilai di dalam `useRef` tidak akan memicu render ulang (*re-render*) pada komponen.

Manfaat menggunakan `useRef`:

1. Mengakses DOM Secara Langsung

Ini adalah penggunaan yang paling umum. Di React, kita biasanya membiarkan React yang mengatur DOM. Tapi terkadang, kita perlu "menyentuh" elemen JSX secara manual, misalnya untuk:

- Fokus pada input secara otomatis.
- Mengukur dimensi sebuah elemen (tinggi/lebar).
- Mengintegrasikan library pihak ketiga yang bukan berbasis React (seperti Google Maps atau Chart.js).

2. Menyimpan Nilai yang "Tahan Render"

Jika menyimpan data di `useState`, setiap kali data berubah, layar akan di-update. Tapi jika Anda butuh menyimpan variabel yang nilainya tetap ada selama siklus hidup komponen tanpa ingin memicu render ulang, `useRef` adalah solusinya.

```
const myRef = useRef(initialValue);

// Cara akses:
console.log(myRef.current);

// Cara ubah:
myRef.current = newValue;
```

Kapan harus memilih atau menggunakan Hook:

- Gunakan `useState` jika datanya harus tampil dan berubah di layar.
- Gunakan `useEffect` jika ingin "keluar" dari ekosistem React (interaksi dengan browser, server, atau timer).
- Gunakan `useContext` hanya jika props drilling sudah terasa sangat mengganggu (lebih dari 3-4 lapis komponen).
- Gunakan `useMemo` hanya jika mendeteksi aplikasi melambat (optimasi dini seringkali tidak perlu).
- Gunakan `useRef` paling sering untuk `.focus()`, `.scroll()`, atau integrasi library grafik.

Perbandingan Hooks secara umum:

Hook	Fungsi Utama	Memicu Re-render
<i>useState</i>	Data dinamis	Ya
<i>useEffect</i>	Efek samping	Tidak langsung
<i>useContext</i>	Data global	Ya
<i>useRef</i>	Referensi / cache	Tidak
<i>useMemo</i>	Optimasi nilai	Tidak

Tabel 5.3 Perbandingan antara setiap Hook

5.3 Praktikum

Pada bagian praktikum ini, pengembangan aplikasi dilanjutkan dari latihan pada bab sebelumnya dengan memfokuskan pada implementasi fitur pencarian buku sederhana. Praktikum ini dirancang untuk memperkuat pemahaman mengenai penggunaan *state*, *props*, dan *hooks* sebagai konsep inti dalam pengelolaan data dan interaksi pada aplikasi React Native.

Melalui studi kasus ini, data buku akan dikelola menggunakan *state*, dikirim antar komponen melalui *props*, serta diproses secara dinamis dengan memanfaatkan *hooks* seperti *useState* dan *useMemo*. Pendekatan ini mencerminkan pola pengembangan aplikasi nyata, di mana perubahan input pengguna secara langsung memengaruhi tampilan dan hasil data yang ditampilkan.

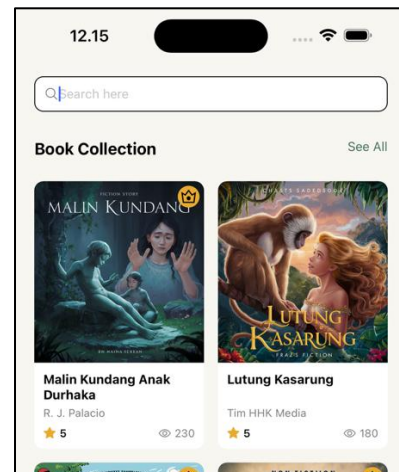
Langkah 1: Menambahkan state dan hooks pada RFC SearchPage

Pada RFC SearchPage, tambahkan dua buah state untuk inputan search dan daftar buku seperti dibawah ini:

Komponen Parent: SearchPage

```
export default function SearchPage() {
  const [search, setSearch] = useState("");
  const [books, setBooks] = useState(ListBook);

  return (
    <SafeAreaView style={styles.container}>
      <StatusBar barStyle={"dark-content"} />
      <SearchBar value={search} setValue={setSearch} />
      <ScrollView style={{ flex: 1 }}>
        <BookCollection books={books} />
      </ScrollView>
    </SafeAreaView>
  );
}
```



Gambar 5.3 Output Program SearchPage

State Management

Menambahkan state search dan books dengan memanfaatkan *useState* sebagai media penyimpanan data dalam bentuk Array *Destructuring*:

```
const [search, setSearch] = useState("");
const [books, setBooks] = useState(ListBook);
```

- State search memiliki nilai awal ialah string kosong
- State books memiliki nilai array object yang diambil dari variable ListBook

Komponen Child:

Dalam RFC SearchPage memiliki dua buah komponen child yaitu RFC SearchBar dan BookCollection. Kedua komponen child tersebut memiliki prop yang akan dikirimkan:

```
<SearchBar value={search} setValue={setSearch} />
```

SearchBar: Menerima nilai search dan fungsi untuk mengubahnya

```
<BookCollection books={books} />
```

BookCollection: Menerima array books untuk ditampilkan

1. Pada RFC SearchBar tambahkan prop dan menginisialisasi prop kedalam TextInput:

```
export default function SearchBar({value, setValue}) {
  return (
    <View style={styles.h_container}>
      <View style={style_searchBar.search_container}>
        <Ionicons name="search-outline" size={16} color="gray" />
        <TextInput
          autoFocus
          placeholder="Search here"
          value={value}
          onChangeText={({text})=>setValue(text)} />
      </View>
    </View>
  );
}
```

Props:

{value, setValue}

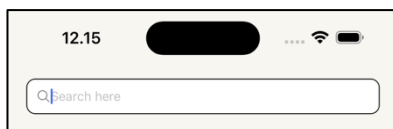
value: String yang berisi teks pencarian saat ini

setValue: Fungsi callback untuk mengupdate nilai pencarian

Fitur TextInput:

```
<TextInput
  autoFocus
  placeholder="Search here"
  value={value}
  onChangeText={({text})=>setValue(text)} />
```

- Properti *autoFocus*: Otomatis fokus saat component muncul
- Properti *placeholder*: Teks placeholder, jika user belum mengisi keyword pada text input
- Properti *value*: Controlled value dari parent
- Properti *onChangeText*: Update state parent



Pada tampilan TextInput, perhatikan terdapat kursor pointer yang akan berkedip secara berkala, hal ini dikarenakan menggunakan autoFocus.

2. Pada RFC BookCollection tidak ada perubahan yang diperlukan, karna pada komponen parent hanya mengganti prop value yang sebelumnya static sekarang diganti dengan hasil return dari useMemo

Langkah 2: Mengoptimisasi data ListBook dengan useMome

```
const BookDataMemori = useMemo(()=>{
  let computedData = [...books];
  if(search){
    computedData = computedData.filter((listData) => {
      return Object.keys(listData).some((key) => {
        try {
          const value = listData[key];
          return value != null && String(value).toLowerCase().includes(search.toLowerCase());
        } catch (error) {
          console.error(`Error processing key "${key}":`, error);
          return false;
        }
      });
    });
  }
  return computedData;
}, [search, books]);
```

Tambahkan code diatas pada bagian komponen parent RFC SearchPage dibawah state management dan gantilah *prop value* pada RFC BookCollection dengan BookDataMemori:

```
<BookCollection books={BookDataMemori} />
```

Penjelasan:

BookDataMemori adalah memoized computed value yang berisi hasil filtering data buku berdasarkan input pencarian.

Cara Kerja Code:

1. Inisialisasi Data

```
let computedData = [...books];
```

- Membuat *shallow copy* dari array books menggunakan *spread operator* (...)
- Mencegah mutasi langsung pada state original

2. Conditional Filtering

```
if(search) {
  computedData = computedData.filter((listData) => { ... });
}
```

- Hanya filter jika ada input pencarian (search tidak kosong)
- Jika search kosong → return semua data

3. Filter Logic - Object.keys()

```
return Object.keys(listData).some((key) => { ... });
```

- *Object.keys(listData)* : Mengambil semua property names dari object buku. Contoh: ["id", "title", "img", "author", "rating", "views", ...]
- *some()* : Return true jika minimal 1 property cocok dengan pencarian

4. Value Matching Logic

```
try {
  const value = listData[key];
  return value != null &&
    String(value).toLowerCase().includes(search.toLowerCase());
} catch (error) {
  console.error(`Error processing key "${key}":`, error);
  return false;
}
```

}

- *listData[key]*: Ambil nilai dari property (misal: *listData["title"]*)
- *value != null*: Pastikan nilai bukan null atau undefined
- *String(value)*: Convert nilai ke string (handle number, boolean, dll)
- *.toLowerCase()*: Case-insensitive search
- *.includes(search.toLowerCase())*: Cek apakah mengandung kata pencarian
- *Try-catch*: Handle error jika ada property yang tidak bisa diproses

Contoh eksekusi:

Pada data buku misal memiliki data seperti ini:

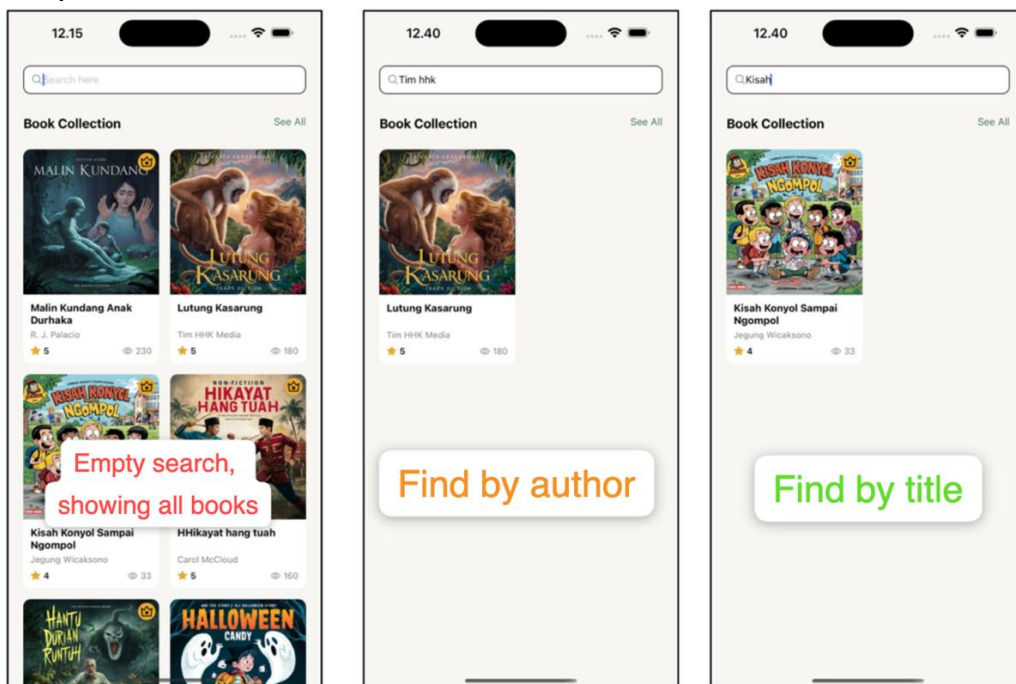
```
{  
  id: 1,  
  title: "Malin Kundang Anak Durhaka",  
  author: "R. J. Palacio",  
  rating: 5,  
  view: 100  
}
```

Proses Filtering:

- Check *property id (1)* : "1".includes("malin") → ❌
- Check *property title* ("Malin Kundang Anak Durhaka") → "malin kundang anak durhaka".includes("malin") → ✅
- *some()* return true → Buku ini masuk hasil

Langkah 3: Mencari item dalam RFC SearchPage

Setelah menambahkan State Management dan Hooks pada RFC SearchPage maka, aplikasi dapat mencari berdasarkan atribut yg dimiliki oleh ListBook seperti mencari berdasarkan author ataupun title.



Gambar 5.4 Tampilan Akhir dari program SearchPage

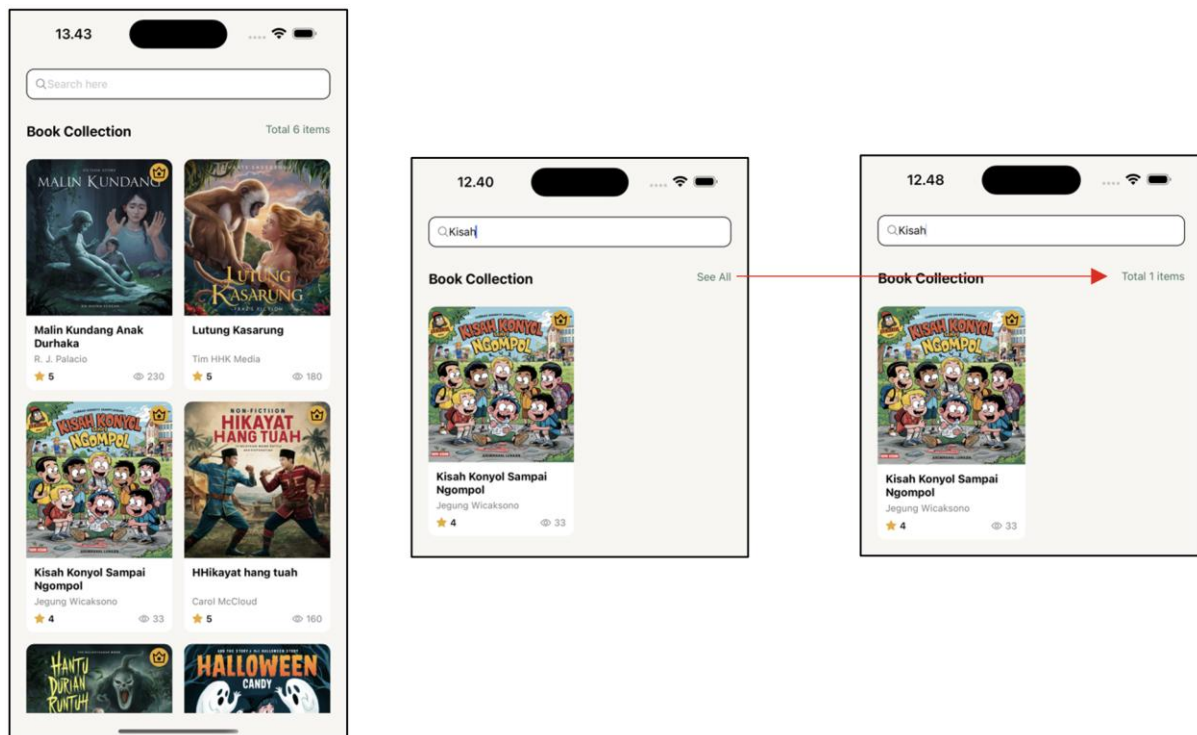
5.4 Tugas Latihan Praktikum

Pada tugas praktikum ini, mahasiswa diminta untuk melanjutkan pengembangan fitur pencarian buku yang telah dibuat pada sesi praktikum sebelumnya dengan menambahkan pengolahan data yang lebih dinamis dan efisien. Fokus utama tugas ini adalah penerapan Hooks pada React Native, khususnya useState, useEffect, dan useMemo, dalam menangani perhitungan data dan pengelolaan tampilan berbasis kondisi.

Latihan 1: Menghitung total buku yang tersedia

Latihan pertama adalah menghitung total data buku yang tersedia maupun yang telah difilter melalui fitur pencarian. Perhitungan ini harus dikelola menggunakan state dan diperbarui secara otomatis ketika terjadi perubahan data, sehingga jumlah buku yang ditampilkan selalu sinkron dengan kondisi aplikasi.

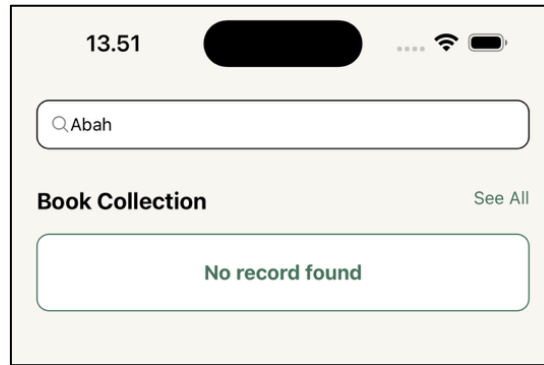
Dalam contoh dibawah ini, tulisan Text yang sebelumnya adalah "See All" menjadi "Total x item".



Gambar 5.5 Tampilan Tugas Latihan 1

Latihan 2: Membuat view untuk menampilkan Data Tidak Ditemukan

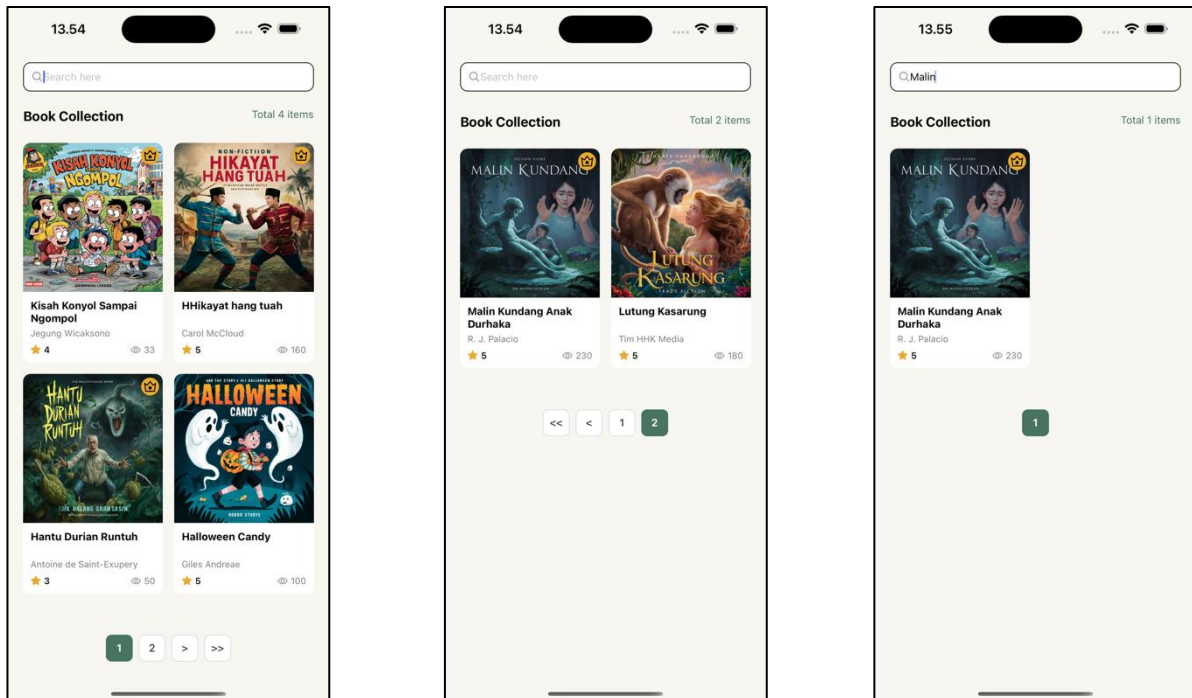
Setiap membangun sebuah aplikasi selalu ada kondisi jika data yang dicari itu tidak ditemukan, pada latihan ke dua ini, buatlah sebuah view bertuliskan "No record found", jika data yang dicari tidak ada.



Gambar 5.6 Tampilan Tugas Latihan 2

Latihan 3: Membuat view paging

Mengimplementasikan fitur paging (*pagination*) pada komponen daftar buku. Paging harus memiliki limit data per halaman yang dapat diatur, serta menampilkan maksimal lima nomor halaman dalam satu waktu. Apabila jumlah data buku melebihi 100 item, maka sistem paging harus secara otomatis menampilkan lima nomor halaman terakhir sebagai representasi halaman akhir.



Gambar 5.6 Tampilan Tugas Latihan 3

Latihan 4: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 3, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project react expo yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository "**PPB-Praktikum-NPM**", namun *commit* dengan nama "**Tugas Praktikum-5**".

Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).